

Rezolvări detaliate - Seminare KR

Rezolvarea problemelor prin căutare

Seminar 1, Seminar 2 și Seminar 3 - explicate pentru examen

Ce conține documentul:

- rezolvări explicate pentru probleme de tip traversare/transport;
- exemple de euristici admisibile, consistente și neadmisibile;
- A* și IDA* pe graful cu trenuri;
- Minimax, Alpha-Beta și jocuri: X și O, 2048, Ultimate X și O, Achi, Three Men's Morris, Connect-4, Alquerque, Fox and Hounds;
- șabloane de răspuns pe care le poți reproduce la examen.

Material de recapitulare pentru examen

Contents

1	Seminar 1 - probleme de traversare si transport	2
1.1	Tigroaica si cei 3 pui	2
1.2	Soti gelosi	2
1.3	Politisti si prizonieri	3
2	Seminar 2 - euristici, A* si IDA*	5
2.1	Demonstreaza ca orice euristica consistenta este admisibila	5
2.2	Turnurile din Hanoi - exemple de euristici	5
2.3	Blocuri cu greutate	6
2.4	8-puzzle	7
2.5	Mersul trenurilor - A*	7
3	Seminar 3 - Minimax, Alpha-Beta si jocuri	10
3.1	Arborele Minimax - intelegere de baza	10
3.2	X si 0 - aprofundare	12
3.3	Jocul 2048	12
3.4	Minimax - relatii intre valorile nodurilor	13
3.5	X si 0 Ultimate	14
3.6	Achi si Three Men's Morris	14
3.7	Connect-4	15
3.8	Alquerque	15
3.9	Fox and Hounds	16
4	Fisa rapida pentru examen	16

1 Seminar 1 - probleme de traversare si transport

1.1 Tigroaica si cei 3 pui

Enunt pe scurt

Avem o tigroaica T , doi pui de tigru obisnuiti A, B si un pui de leopard L . Leopardul nu poate fi lasat singur cu niciun pui de tigru daca mama nu este acolo. Tigroaica poate transporta un singur pui o data.

Reprezentare ca problema de cautare

O stare poate fi scrisa ca:

$$s = (\text{poz}(T), \text{poz}(L), \text{poz}(A), \text{poz}(B)),$$

unde fiecare pozitie este 0 pentru adapostul vechi si 1 pentru adapostul nou.

Starea initiala:

$$s_0 = (0, 0, 0, 0).$$

Starea scop:

$$s_f = (1, 1, 1, 1).$$

O stare este sigura daca pe nicio parte fara mama nu exista simultan leopardul si cel putin un pui de tigru.

Solutia minima

Notam cu $\rightarrow$ drumul spre adapostul nou si cu $\leftarrow$ intoarcerea spre adapostul vechi.

Pas	Mutare
1	T duce leopardul L la noul adapost: $T + L \rightarrow$
2	T revine singura: $T \leftarrow$
3	T duce puiul A : $T + A \rightarrow$
4	T aduce leopardul inapoi: $T + L \leftarrow$
5	T duce puiul B : $T + B \rightarrow$
6	T revine singura: $T \leftarrow$
7	T duce leopardul L : $T + L \rightarrow$

Raspuns: numarul minim este 7 drumuri.

De ce nu merge mai repede?

Daca ducem un pui de tigru si apoi mama pleaca dupa altul, leopardul nu poate ramane pe aceeasi parte cu puiul fara mama. De aceea leopardul trebuie dus, adus inapoi, iar apoi dus din nou la final. Aceasta intoarcere suplimentara forteaza cele 7 drumuri.

1.2 Soti gelosi

Model general

Avem N cupluri (H_i, W_i) si M magari, deci o traversare poate duce cel mult M persoane. Magarii nu se intorc singuri, deci dupa o urcare cineva trebuie sa revina cu magarii, in afara ultimei curse.

O stare poate fi:

$$s = (S, \text{poz}),$$

unde S este multimea persoanelor ramase jos, iar $\text{poz} \in \{\text{jos}, \text{sus}\}$ indica unde se afla magarii.

Conditie de siguranta: pentru orice sotie W_i , daca W_i este pe o parte si sotul H_i nu este acolo, atunci pe acea parte nu trebuie sa fie niciun alt barbat.

La examen, nu trebuie sa memorezi doar secventa. Trebuie sa arati ca verifici mereu conditia: nicio femeie nu ramane cu alt barbat in absenta sotului ei.

Cazul $N = 3, M = 2$

Notăm cu H_i sotul și cu W_i soția din cuplul i .

O soluție minimă are 11 drumuri:

Pas	Mutare
1	$W_1, W_3 \rightarrow$
2	$W_3 \leftarrow$
3	$W_2, W_3 \rightarrow$
4	$W_2 \leftarrow$
5	$H_1, H_3 \rightarrow$
6	$H_3, W_3 \leftarrow$
7	$H_2, H_3 \rightarrow$
8	$W_1 \leftarrow$
9	$W_2, W_3 \rightarrow$
10	$W_3 \leftarrow$
11	$W_1, W_3 \rightarrow$

Răspuns: 11 drumuri.

Cazul $N = 4, M = 2$

Pentru patru cupluri și doar doi magari, problema nu are soluție în modelul clasic al sotelor gelosi. Intuitiv, capacitatea 2 este prea mică pentru a transfera suficientă informație/persoane fără să creezi o configurație în care o soție rămâne cu alt bărbat fără sotul ei.

Răspuns: nu există soluție.

Cazul $N = 4, M = 3$

O soluție minimă are 9 drumuri:

Pas	Mutare
1	$W_1, W_4 \rightarrow$
2	$W_4 \leftarrow$
3	$W_2, W_3, W_4 \rightarrow$
4	$W_2 \leftarrow$
5	$H_1, H_3, H_4 \rightarrow$
6	$H_3, W_3 \leftarrow$
7	$H_2, H_3 \rightarrow$
8	$W_4 \leftarrow$
9	$W_2, W_3, W_4 \rightarrow$

Răspuns: 9 drumuri.

1.3 Politisti și prizonieri**Model**

Avem N_{Po} politisti, N_{Pr} prizonieri, mașina cu M locuri. Prizonierii nu pot conduce, deci fiecare cursă trebuie să conțină cel puțin un politist. Prizonierii nu trebuie să fie mai mulți decât politistii în mașina sau în afara facilității.

O stare poate fi:

$$s = (Po_{in}, Pr_{in}, pozMasina).$$

Starea initiala:

$$(NPo, NPr, in).$$

Starea scop:

$$(0, 0, out).$$

Cazul $NPo = 3, NPr = 2, M = 2$

Notam P = politist, R = prizonier.

Pas	Mutare
1	$P + R \rightarrow$
2	$P \leftarrow$
3	$P + P \rightarrow$
4	$P \leftarrow$
5	$P + P \rightarrow$
6	$P \leftarrow$
7	$P + R \rightarrow$

Raspuns: cel mai scurt drum are 7 curse.

Conditii pentru existenta solutiei

Pentru cazul uzual cu $M = 2$ si prizonieri prezenti, o conditie naturala de solvabilitate este:

$$NPo \geq NPr.$$

Daca la final ar fi mai multi prizonieri decat politisti afara, configuratia finala ar fi nesigura. Pentru $M = 2$, cand $NPo \geq NPr \geq 1$, numarul minim de drumuri este:

$$2(NPo + NPr) - 3.$$

Verificare pentru $NPo = 3, NPr = 2$:

$$2(3 + 2) - 3 = 7.$$

2 Seminar 2 - euristici, A* si IDA*

2.1 Demonstrează că orice euristica consistentă este admisibilă

Definiții

O euristica este **consistentă** dacă pentru orice muchie (n, n') :

$$h(n) \leq c(n, n') + h(n').$$

O euristica este **admisibilă** dacă:

$$h(n) \leq h^*(n),$$

unde $h^*(n)$ este costul real minim de la n la o stare scop.

Demonstratie

Fie un drum optim de la n la scop:

$$n = n_0 \rightarrow n_1 \rightarrow \dots \rightarrow n_k = G.$$

Din consistentă:

$$h(n_0) \leq c(n_0, n_1) + h(n_1),$$

$$h(n_1) \leq c(n_1, n_2) + h(n_2),$$

și așa mai departe.

Adunând în lant, obținem:

$$h(n_0) \leq c(n_0, n_1) + c(n_1, n_2) + \dots + c(n_{k-1}, G) + h(G).$$

Pentru o stare scop, $h(G) = 0$, deci:

$$h(n) \leq \text{cost}(n \rightarrow G) = h^*(n).$$

Deci euristica este admisibilă.

Important: Consistentă este o condiție mai puternică decât admisibilitatea. Orice euristica consistentă este admisibilă, dar nu orice euristica admisibilă este consistentă.

2.2 Turnurile din Hanoi - exemple de euristici

Context

Avem M tije și N discuri. Putem muta doar discul de sus și nu putem pune un disc mare peste unul mic. Scopul: toate discurile să ajungă pe o altă tijă.

1. Euristica admisibilă și consistentă

O variantă foarte sigură este:

$$h(s) = \begin{cases} 0, & s \text{ este scop,} \\ 1, & s \text{ nu este scop.} \end{cases}$$

Este admisibilă deoarece dintr-o stare non-scop ai nevoie de cel puțin o mutare. Este consistentă deoarece valoarea lui h se poate schimba cu cel mult 1, iar costul oricărei mutări este 1.

2. Euristica admisibila, dar neconsistenta

Alegem o stare speciala s_0 care este la cel putin doua mutari de scop. Definim:

$$h(s) = \begin{cases} 2, & s = s_0, \\ 0, & \text{altfel.} \end{cases}$$

Este admisibila fiindca $h(s_0) = 2 \leq h^*(s_0)$. Nu este consistenta daca exista o mutare de la s_0 la un vecin s_1 cu $h(s_1) = 0$, deoarece:

$$h(s_0) = 2 > 1 + 0 = c(s_0, s_1) + h(s_1).$$

3. Euristica neadmisibila

Un exemplu:

$$h(s) = h^*(s) + 1.$$

Aceasta supraestimeaza mereu costul real, deci nu este admisibila. Daca nu stim h^* , putem spune simplu: $h(s) = 100$ pentru orice stare non-scop, ceea ce sigur supraestimeaza pentru starile aflate la o mutare de scop.

2.3 Blocuri cu greutate

Context

Putem muta blocul din varful unei stive pe orice alta stiva. Costul mutarii este greutatea blocului mutat. Scopul este o configuratie data.

1. Euristica admisibila si consistenta

Definim un bloc ca fiind **corect asezat** daca se afla in stiva corecta si toate blocurile de sub el coincid cu configuratia finala.

O euristica buna:

$$h(s) = \sum_{b \text{ incorect}} w(b),$$

unde $w(b)$ este greutatea blocului.

Este admisibila deoarece fiecare bloc incorect trebuie mutat cel putin o data, iar mutarea lui costa $w(b)$. Este consistenta deoarece o mutare de cost w poate schimba suma cu cel mult w .

2. Euristica admisibila, dar neconsistenta

Ca la Hanoi, alegem o stare speciala s_0 aflata la cost real cel putin 2 fata de scop:

$$h(s) = \begin{cases} 2, & s = s_0, \\ 0, & \text{altfel.} \end{cases}$$

Daca exista un vecin s_1 obtinut printr-o mutare de cost 1, atunci:

$$2 > 1 + 0,$$

asa ca euristica nu este consistenta, dar poate ramane admisibila daca $h^*(s_0) \geq 2$.

3. Euristica neadmisibila

$$h(s) = 2 \cdot \sum_{b \text{ incorect}} w(b).$$

Aceasta poate supraestima costul real, deci este neadmisibila.

2.4 8-puzzle

1. Euristică admisibilă și consistentă

Cea mai folosită:

$$h(s) = \sum_{\text{piesa } i} d_{\text{Manhattan}}(\text{poz}_i, \text{poz}_i^{\text{scop}}).$$

Fiecare mutare deplasează o singură piesă cu o unitate, deci suma distanțelor Manhattan scade cu cel mult 1 pe mutare. De aceea este și admisibilă, și consistentă.

Pentru starea din seminar:

$$\begin{array}{ccc} 2 & X & 7 \\ 5 & 1 & 4 \\ 3 & 8 & 6 \end{array} \rightarrow \begin{array}{ccc} X & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{array}$$

Manhattan pe piese este:

$$1 + 2 + 1 + 1 + 2 + 2 + 3 + 1 = 13.$$

Deci $h_{\text{Manhattan}} = 13$.

2. Euristică admisibilă, dar neconsistentă

Alegem o stare s_0 cu $h^*(s_0) \geq 2$ și definim:

$$h(s) = \begin{cases} 2, & s = s_0, \\ 0, & \text{altfel.} \end{cases}$$

Este admisibilă, dar poate încălca consistența pe o muchie de cost 1 către o stare cu euristică 0.

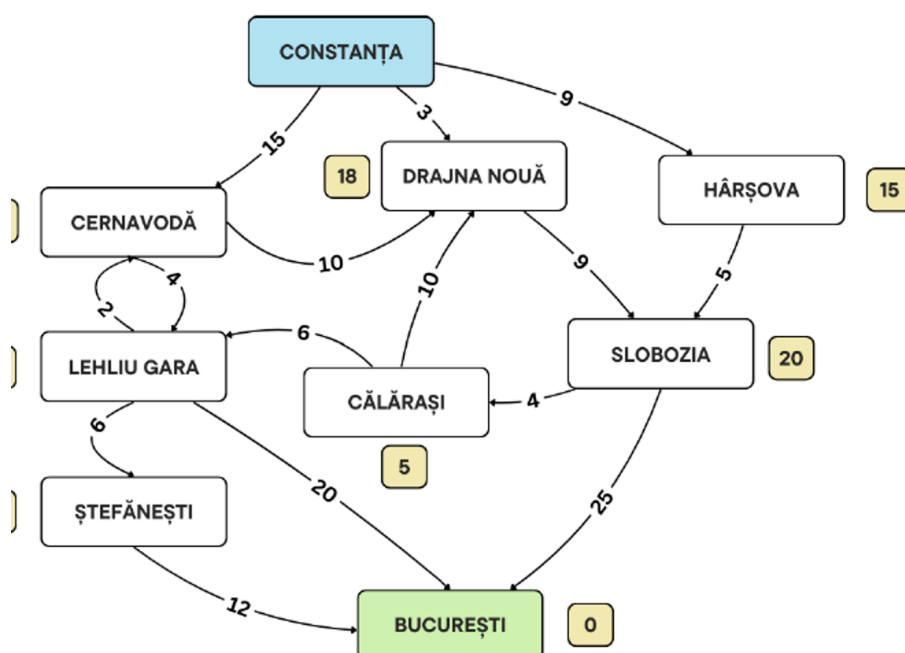
3. Euristică neadmisibilă

$$h(s) = 2 \cdot h_{\text{Manhattan}}(s).$$

Poate supraestima costul real, deci nu este admisibilă.

2.5 Mersul trenurilor - A*

caș, ca în imagine:



putem estima timpul drumului într-un mod admisibil dacă nu am

1. Cum estimăm admisibil fără informația CFR?

Putem folosi:

- distanța geografică în linie dreaptă împartită la viteza maximă posibilă a trenului;
- sau euristica triviale $h(n) = 0$.

$h(n) = 0$ este mereu admisibilă, dar nu este informativă; transformăm A^* în UCS.

2. Primul cost estimat f

Folosim:

$$f(n) = g(n) + h(n).$$

Din Constanta se generează:

- Harsova: $g = 9$, $h = 15$, deci $f = 24$;
- Cernavoda: $g = 15$, $h = 10$, deci $f = 25$;
- Drajna Noua: $g = 3$, $h = 18$, deci $f = 21$.

Calarasi este generat prima dată prin Constanta $\rightarrow$ Drajna Noua $\rightarrow$ Slobozia $\rightarrow$ Calarasi:

$$g = 3 + 9 + 4 = 16, \quad h = 5, \quad f = 21.$$

Stefanesti este generat prima dată prin Constanta $\rightarrow$ Cernavoda $\rightarrow$ Lehliu Gara $\rightarrow$ Stefanesti:

$$g = 15 + 4 + 6 = 25, \quad h = 6, \quad f = 31.$$

3. Ce valori admisibile poate avea h pentru Slobozia și Drajna Noua?

Pentru admisibilitate trebuie:

$$0 \leq h(n) \leq h^*(n).$$

Din Slobozia spre Bucuresti există drum direct de cost 25, iar acesta este cel mai bun în graful dat, deci:

$$0 \leq h(\text{Slobozia}) \leq 25.$$

Din Drajna Noua, drumul evident este:

$$\text{Dragna Noua} \rightarrow \text{Slobozia} \rightarrow \text{Bucuresti},$$

cu cost:

$$9 + 25 = 34.$$

Deci o valoare admisibilă trebuie să respecte:

$$0 \leq h(\text{Dragna Noua}) \leq 34.$$

4. Primul lanț soluție determinat de A^*

Calculăm pas cu pas:

Pas	OPEN după expansiune	Nod ales
Start	Cernavoda $f = 25$, Dragna $f = 21$, Harsova $f = 24$	Dragna
1	Harsova 24, Cernavoda 25, Slobozia 32	Harsova
2	Cernavoda 25, Slobozia 32	Cernavoda
3	Lehliu 31, Slobozia 32	Lehliu
4	Stefanesti 31, Slobozia 32, Bucuresti 39	Stefanesti
5	Slobozia 32, Bucuresti 37, Bucuresti 39	Slobozia
6	Calarasi 21, Bucuresti 37, Bucuresti 37/39	Calarasi

Un lant solutie optim gasit este:

$$Constanta \rightarrow Cernavoda \rightarrow Lehliu Gara \rightarrow Stefanesti \rightarrow Bucuresti,$$

cu cost:

$$15 + 4 + 6 + 12 = 37.$$

Exista si drumul:

$$Constanta \rightarrow Drajna Noua \rightarrow Slobozia \rightarrow Bucuresti,$$

cu acelasi cost:

$$3 + 9 + 25 = 37.$$

Daca apar egalitati in OPEN, primul lant returnat depinde de regula de departajare.

5. IDA* pe exemplu

IDA* face cautari DFS repetate cu prag pe $f = g + h$.

- Prag initial: daca nu avem $h(Constanta)$, putem lua 0 sau o estimare admisibila proprie.
- La fiecare DFS, taiem orice ramura cu $f > prag$.
- Daca nu gasim scop, pragul urmator devine cel mai mic f care a depasit pragul curent.

Sucesiunea pragurilor relevante, din valorile generate de A*, este:

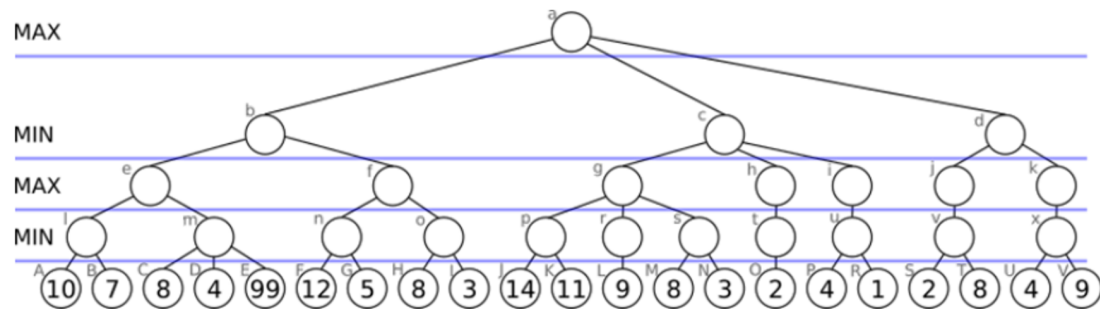
$$21, 24, 25, 31, 32, 37.$$

La prag 37 se gaseste pentru prima data un drum catre Bucuresti.

3 Seminar 3 - Minimax, Alpha-Beta și jocuri

3.1 Arborele Minimax - înțelegere de bază

1 Arborele Minimax - înțelegere de bază



1. Care e ordinea de evaluare a nodurilor?

Ordinea de evaluare

Dacă parcurgem standard de la stânga la dreapta, frunzele se evaluează în ordinea:

$$A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, R, S, T, U, V.$$

Lipsește litera Q în notarea din imagine.

Completarea arborelui cu valori

Nivelurile sunt: radacina MAX, apoi MIN, apoi MAX, apoi MIN, apoi frunze.

Noduri MIN de jos:

$$\begin{aligned} l &= \min(10, 7) = 7, & m &= \min(8, 4, 99) = 4, \\ n &= \min(12, 5) = 5, & o &= \min(8, 3) = 3, \\ p &= \min(14, 11) = 11, & r &= 9, & s &= \min(8, 3) = 3, \\ t &= 2, & u &= \min(4, 1) = 1, & v &= \min(2, 8) = 2, & x &= \min(4, 9) = 4. \end{aligned}$$

Noduri MAX:

$$\begin{aligned} e &= \max(l, m) = \max(7, 4) = 7, \\ f &= \max(n, o) = \max(5, 3) = 5, \\ g &= \max(p, r, s) = \max(11, 9, 3) = 11, \\ h &= 2, & i &= 1, & j &= 2, & k &= 4. \end{aligned}$$

Noduri MIN de nivel 1:

$$\begin{aligned} b &= \min(e, f) = \min(7, 5) = 5, \\ c &= \min(g, h, i) = \min(11, 2, 1) = 1, \\ d &= \min(j, k) = \min(2, 4) = 2. \end{aligned}$$

Radacina MAX:

$$a = \max(b, c, d) = \max(5, 1, 2) = 5.$$

Variația principală

La fiecare nivel alegem copilul care determină valoarea nodului:

$$a \rightarrow b \rightarrow f \rightarrow n \rightarrow G.$$

Valoarea finală este 5.

Urmatoarea mutare pentru MAX

Radacina este MAX si alege copilul cu valoare maxima. Copiii sunt:

$$b = 5, \quad c = 1, \quad d = 2.$$

Deci MAX alege mutarea catre b .

Modificarea unei frunze care nu este pe variatia principala

O varianta: modificam frunza B din 7 in 0. Atunci:

$$\begin{aligned} l &= \min(10, 0) = 0, \\ e &= \max(l, m) = \max(0, 4) = 4, \\ b &= \min(e, f) = \min(4, 5) = 4. \end{aligned}$$

Radacina devine:

$$a = \max(4, 1, 2) = 4.$$

Deci valoarea radacinii s-a schimbat din 5 in 4 fara sa modificam frunza din variatia principala.

Interval pentru frunza G din variatia principala

Fie valoarea lui G egala cu x .

$$n = \min(12, x).$$

Ca n sa aleaga tot G , avem nevoie de:

$$x \leq 12.$$

Ca f sa aleaga tot n in loc de $o = 3$, avem nevoie de:

$$\min(12, x) \geq 3.$$

Ca b sa aleaga tot f in loc de $e = 7$, avem nevoie de:

$$\min(12, x) \leq 7.$$

Deci intervalul este aproximativ:

$$3 \leq x \leq 7.$$

Daca vrem sa evitam egalitatile si ambiguitatea de departajare, folosim:

$$3 < x < 7.$$

Alpha-Beta pe arbore

Parcurgem de la stanga la dreapta. Dupa ce se evalueaza subarborele b , radacina MAX are:

$$\alpha = 5.$$

In subarborele c , dupa evaluarea lui $h = 2$, nodul c este MIN si are deja o valoare $2 \leq \alpha = 5$. Deci restul lui c se poate reteza: nodul i si frunzele sale nu se mai expandeaza.

In subarborele d , dupa ce $j = 2$, nodul d este MIN si are $2 \leq \alpha = 5$, deci se reteaza k si frunzele lui.

O lista de frunze/noduri care pot fi retezate in aceasta ordine:

$$N, \quad i \text{ cu } P, R, \quad T, \quad k \text{ cu } U, V.$$

Se poate reteza un nod din variatia principala?

Nu, in Alpha-Beta corect, nodurile de pe variatia principala trebuie evaluate pentru a determina valoarea finala. Retezarea elimina doar ramuri care nu mai pot influenta decizia finala.

Reordonarea succesorilor

Da. Pentru mai multe rețezări:

- la nodurile MAX, punem mai întâi copiii cu valori mari;
- la nodurile MIN, punem mai întâi copiii cu valori mici.

Aceasta produce limite α, β mai strânse mai devreme.

3.2 X și 0 - aprofundare

Starea 1, MAX = 0

Tabla este:

0	x	—
x	0	x
—	—	—

Este rândul lui 0. Mutarea castigatoare este colțul dreapta-jos:

0	x	—
x	0	x
—	—	0

Se formează diagonală principală 0, 0, 0.

Valoarea minimax: +1 pentru MAX. Alpha-Beta va rețeza orice ramură ulterioară după ce găsește această mutare castigatoare, dacă ordinea mutărilor o pune prima.

Starea 2, MAX = x

Tabla este:

0	x	x
—	0	—
x	0	—

Este rândul lui x . Dacă x nu joacă în colțul dreapta-jos, 0 câștigă imediat pe diagonală. Deci mutarea obligatorie este:

0	x	x
—	0	—
x	0	x

După aceea jocul se termină remiza cu joc optim.

Valoarea minimax: 0 pentru MAX, adică remiza. Mutarea bună este blocarea diagonalei în dreapta-jos.

3.3 Jocul 2048

Formalizare

O stare este matricea 4×4 :

$$s = (a_{ij})_{1 \leq i, j \leq 4},$$

unde fiecare $a_{ij} \in \{0, 2, 4, 8, \dots, 2048, \dots\}$, iar 0 înseamnă casuta liberă.

Starea inițială: tabla cu două plăcuțe de valoare 2 în poziții aleatoare.

Stare scop:

$$\exists i, j \text{ astfel încât } a_{ij} = 2048.$$

Stare de eșec: tabla plină și nu există nicio mutare legală.

Se poate aplica Minimax?

Minimax simplu nu este modelul cel mai potrivit, deoarece după fiecare mutare apare aleator o piesă nouă. Nu avem doar MAX și MIN, ci avem MAX + sansă. Modelul corect este **ExpectiMax**.

Euristica optimista catre scop

O euristica simpla: numarul minim de dublari necesare pentru a ajunge de la cea mai mare placa m la 2048:

$$h(s) = \max(0, \log_2(2048) - \log_2(m)).$$

Aceasta este optimista, deoarece presupune ca putem combina perfect placutele necesare.

Expected value la adancime 3

La adancime 3, alternam:

$$MAX \rightarrow Chance \rightarrow MAX$$

Valoarea se calculeaza recursiv:

$$V_{MAX}(s) = \max_a V(Succ(s, a)),$$

$$V_{Chance}(s) = \sum_{s'} P(s' | s) V(s').$$

Daca toate aparitiile sunt considerate egal probabile, atunci nodul Chance face media simpla.

Probabilitati 2/4

Daca sunt k casute libere, atunci pentru fiecare casuta libera:

$$P(2 \text{ in casuta}) = \frac{0.9}{k}, \quad P(4 \text{ in casuta}) = \frac{0.1}{k}.$$

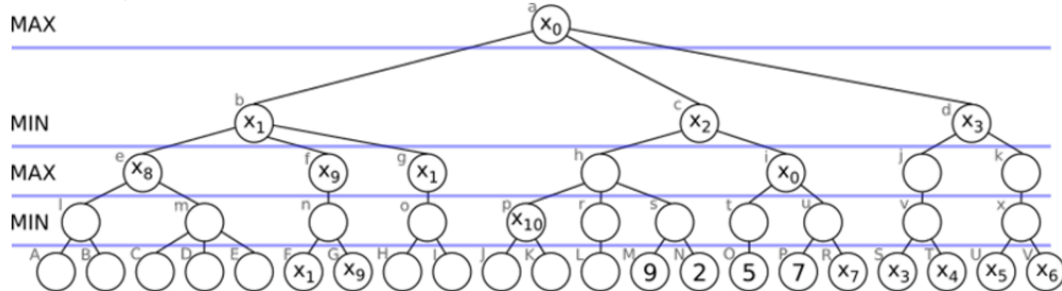
Atunci:

$$V_{Chance}(s) = \sum_{\text{celule libere } c} \left(\frac{0.9}{k} V(s_{c,2}) + \frac{0.1}{k} V(s_{c,4}) \right).$$

3.4 Minimax - relatii intre valorile nodurilor

informația dată despre arbore?

Observație: frunzele sunt notate cu litere mari iar nodurile interne cu litere mici.



2

Raspunsuri

Afirmatiile sigur adevarate sunt:

a) adevarat, b) adevarat.

Celelalte nu sunt garantate de informatia din arbore.

Justificare pentru a)

In partea dreapta, nodul d este MIN si are valoarea x_3 . Subarborele sau contine valori care forteaza:

$$x_3 \leq x_4, \quad x_3 \leq x_5, \quad x_3 \leq x_6.$$

Deci afirmatia a) este sigur adevarata.

Justificare pentru b)

În subarborele notat cu $i = x_0$, valoarea este:

$$x_0 = \max(5, \min(7, x_7)).$$

De aici rezulta:

$$5 \leq x_0 \leq 7.$$

Deci b) este sigur adevărată.

De ce celelalte nu sunt sigure?

- c) x_2 poate fi mai mic decât x_0 , deoarece $x_2 = \min(h, x_0)$.
- d) x_1 nu trebuie neapărat să fie egal cu x_9 ; poate fi determinat de alt copil.
- e) condiția $x_8 > x_9$ nu garantează că nodul g va fi rețezat, deoarece limita α de la rădăcina poate să nu permită această rețezare.
- f) condiția $x_{10} < 9$ nu garantează rețezarea lui N ; depinde de limita α acumulată în acel punct.

3.5 X și 0 Ultimate**Stare**

O stare trebuie să țină:

- cele 9 table mici 3×3 ;
- starea fiecărei table mici: câștigată de X, câștigată de 0, remiză sau activă;
- jucătorul la mutare;
- tabla mică în care este obligat să joace următorul jucător, dacă există.

$$s = (B_{mare}, status_{table}, player, forcedBoard).$$

Acțiuni

Dacă *forcedBoard* este validă și nu este plină/câștigată, acțiunile sunt casutele libere din acea tablă mică. Dacă tabla forțată este terminată, acțiunile sunt toate casutele libere din tablele mici neterminate.

Scop

Un jucător câștigă dacă a câștigat trei table mici care formează linie, coloană sau diagonală pe tabla mare.

Minimax

Se poate aplica Minimax deoarece jocul este determinist, cu doi jucători, complet observabil și de sumă zero. În practică se folosește adâncime limitată plus funcție de evaluare.

3.6 Achi și Three Men's Morris**Stare generală**

Pentru ambele jocuri, o stare poate fi:

$$s = (board, player, phase),$$

unde:

- *board* reține piesele pe cele 9 poziții;
- *player* este jucătorul care urmează;
- *phase* este faza de plasare sau faza de mutare.

Stare scop

Un jucator are trei piese pe aceeasi linie, coloana sau diagonala.

Actiuni

In faza de plasare: pui o piesa pe o pozitie libera. In faza de mutare: muti o piesa proprie pe un punct adiacent liber, de-a lungul unei linii.

Euristica optimista

Pentru MAX:

$$h(s) = 3 - \max_{linie} \#(pieseMAX \text{ pe linie}).$$

Daca MAX are deja doua piese pe o linie, $h = 1$; daca are una, $h = 2$. Este o estimare optimista, deoarece ignora blocajele adversarului.

Minimax si Alpha-Beta

Se aplica Minimax deoarece jocurile sunt deterministe, cu doi jucatori si informatie completa. Alpha-Beta poate elimina ramuri fara a schimba rezultatul Minimax.

3.7 Connect-4

1. Cine urmeaza sa mute?

Daca numarul de piese rosii si galbene este egal, muta primul jucator. Daca primul jucator are cu o piesa mai mult, muta al doilea.

2. Cati succesori are starea?

Numarul de succesori este numarul de coloane care nu sunt pline. Maximum:

$$b_{max} = 7.$$

3. Functii de evaluare a frunzelor

Exemple:

- $+\infty$ pentru victorie MAX, $-\infty$ pentru victorie MIN;
- numarul de linii potentiale de 4 pentru MAX minus cele pentru MIN;
- bonus pentru trei piese consecutive cu un loc liber;
- bonus pentru controlul coloanelor centrale;
- penalizare daca adversarul are amenintare imediata.

3.8 Alquerque

Stare

$$s = (board, player),$$

unde tabla are 25 puncte, iar fiecare punct poate fi gol, piesa MAX sau piesa MIN.

Actiuni

Daca exista capturi, jucatorul este obligat sa faca o captura. O actiune poate fi:

- mutare simpla pe un punct adiacent liber;
- secventa de capturi consecutive cu aceeasi piesa.

Stare scop

Un jucator castiga daca adversarul nu mai are piese sau nu mai are mutari legale.

Evaluare

$$Eval(s) = 10 \cdot (pieseMAX - pieseMIN) + mobilitateMAX - mobilitateMIN.$$

Se poate adauga bonus pentru capturi disponibile.

3.9 Fox and Hounds**Stare**

$$s = (pozFox, pozH_1, pozH_2, pozH_3, pozH_4, player).$$

Actiuni

- Vulpea se misca diagonal in orice directie, un pas, pe o casuta libera.
- Cainii se misca doar diagonal inainte, un pas, pe o casuta libera.

Stare scop

Vulpea castiga daca ajunge pe randul de start al cainilor. Cainii castiga daca vulpea nu mai are nicio mutare legala.

Funcție de evaluare

Pentru MAX = vulpea:

$$Eval(s) = distanta_progres_vulpe - \lambda \cdot blocare,$$

unde *blocare* poate fi numarul de mutari legale ale vulpii lipsa fata de maximum. O euristica simpla favorizeaza vulpea cu cat este mai aproape de randul tinta.

4 Fisa rapida pentru examen**Traversari**

- Stare = cine este pe fiecare parte + pozitia barcii/masinii/magarilor.
- Actiune = grup valid care traverseaza.
- Verifici siguranta dupa fiecare mutare, pe ambele parti.

Euristici

- Admisibila: $h(n) \leq h^*(n)$.
- Consistenta: $h(n) \leq c(n, n') + h(n')$.
- Consistenta implica admisibilitate.
- Pentru a arata neconsistenta, gasesti o muchie unde diferenta euristicilor este prea mare.

A*

- $f(n) = g(n) + h(n)$.
- Alegi mereu nodul cu f minim.
- Daca apar egalitati, rezultatul poate depinde de regula de departajare.

IDA*

- Face DFS cu prag pe f .
- Dacă nu găsește scop, pragul devine cel mai mic f care a depășit pragul anterior.

Minimax/Alpha-Beta

- MAX ia maximumul copiilor.
- MIN ia minimumul copiilor.
- Alpha-Beta nu schimbă valoarea Minimax.
- Tai când $\alpha \geq \beta$.
- Ordonare bună: MAX vede întâi valori mari, MIN vede întâi valori mici.

ExpectiMax

- MAX: maxim.
- Chance: medie ponderată.
- Se folosește când mediul are aleatoriu, ca în 2048.